UltraScale および UltraScale+ アーキテクチャ DSP リソース

2025.2

概要

この演習では、UltraScale™ および UltraScale+™ アーキテクチャの DSP リソースについて説明します。

このラボの所要時間は約 45 分です。

CloudShare ユーザーのみ

ラボを完了するための試行回数は 3 回まで提供されており、各ラボの完了に割り当てられている時間は想定完了時間の 2 倍です。タイマーが開始されると、一時停止することはできません。新しい試行を開始するたびに前回の試行はリセットされ、過去の試行での作業内容は保存されません。

目的

このラボを完了すると、次のことができるようになります:

- [Language Templates] を使用した 2 乗関数の作成
- デザイン内の DSP リソースを確認およびレビューします

はじめに

UltraScale および UltraScale+ FPGA には DSP48E2 スライスが含まれています。この DSP48E2 には、以前のアーキテクチャ世代と比較して追加機能が備わっています。

- より広い乗算器 (27 x 18)
- より広幅のプリ加算器、D、および AD レジスタ (27 ビット)
- プリ加算器の柔軟性の向上
- 乗算器を使用して、一部の入力およびプリ加算器の結果を 2 乗する機能

© Copyright 2026 Advanced Micro Devices, Inc.

- W マルチプレクサーによって制御される ALU への追加オペランド (48 ビット RND 定数を含む)
- ワイド XOR 機能

AMD では、可能な限り DSP などの専用リソースを使用することを推奨しています。

ラボ環境の理解

このコースで提供されるラボおよびデモは、Linux® プラットフォーム上で実行するように設計されています。

1 つの環境変数 TRAINING_PATH が必要です。これはラボ ファイルの場所を指します。この変数は、CloudShare/CustEd_VM 環境にあらかじめ設定されています。

一部のツールはこの環境変数を直接使用できますが (つまり、$TRAINING_PATH が認識されて自動的に展開されます)、一部のツールでは手動で展開する必要があります (CloudShare/CustEd_VM 環境の場合は /home/amd/training)。各ツールでの対応手順については、演習手順書に記載されています。その他の環境では、スクリプトが正しく動作するためにこの変数を定義する必要があります。

Vivado™ Design Suite および Vitis™ Unified IDE は、多くの演習で使用される Tcl 環境を提供します。どちらのツールも起動時には、以前のツール起動時の proc や変数が一切残っていないクリーンな Tcl 環境で起動します。

Tcl スクリプトを source したり手動で Tcl 変数を設定した後にツールを終了した場合、ツールを再度開いたときに、Tcl スクリプトを再度 source し、演習に必要な変数を設定する必要があります。これはターミナル ウィンドウでも同様であり、新しいターミナルを開くとすべての変数設定がクリアされます。

表記規則

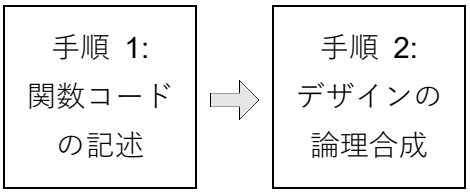
各種引数の使用方法を説明するために、正式な表記規則が使用されています。以下は、主に使用される記号の一部です。

シンボル	説明	例	説明
<text>	フィールドを示します	`cd <dir>`	`<dir>` はディレクトリ名を表します。< および > 記号は入力しません。移動先のディレクトリが `XYZ` の場合、環境に `cd XYZ` と入力します。

© Copyright 2026 Advanced Micro Devices, Inc.

シンボル	説明	例	説明
[text]	オプションの引数を示します	`ls [ | more]`	これは ls <Enter> または ls | more <Enter> と解釈できます。最初の例は現在の Linux ディレクトリ内のファイルを一覧表示し、2 番目の例は現在の Linux ディレクトリ内のファイルを一覧表示しますが、さらに出力を more ツールに渡してページ分割します。ここで、パイプ記号 (|) は Linux の演算子です。
|	選択肢を示します	`cmd <ZCU104 | VCK190>`	cmd コマンドは 1 つの引数を取ります。引数には ZCU104 または VCK190 を指定できます。cmd ZCU104 または cmd VCK190 のいずれかを入力します。

全体の流れ



二乗関数のコードの記述 手順 1

[Vivado Design Suite] を起動して提供されたプロジェクトを開き、演習を開始します。ここでは、[Language Templates] から squarer コードを追加します。

© Copyright 2026 Advanced Micro Devices, Inc.

1-1. Vivado Design Suite を起動します。

このタスクの実行方法を覚えていない場合は、Lab Reference Guide の Vivado Design Suite Operations にある「Launching the Vivado Design Suite」トピックを参照してください。

1-2. [Tcl Console] を使用してプロジェクトを解凍します。

1-2-1. **[Tcl Console]** に次のコマンドを入力して、プロジェクトを解凍します:

```
exec unzip $::env(TRAINING_PATH)/dsp_resources/lab/<KCU105 |
SCU035>/verilog.zip -d
$::env(TRAINING_PATH)/dsp_resources/lab/<KCU105 |
SCU035>/verilog
```

1-3. 以下のディレクトリにある squarer.xpr という名前の Vivado Design Suite プロジェクトを開きます。

1-3-1. 次のディレクトリに移動します:

```
$TRAINING_PATH/dsp_resources/lab/<KCU105 | SCU035>/verilog
```

1-3-2. **squarer.xpr** プロジェクトを開きます。

このタスクの実行方法を覚えていない場合は、Lab Reference Guide の Vivado Design Suite Operations にある「Opening a Vivado Design Suite Project」トピックを参照してください。

1-4. [Language Templates] を開き、2 乗関数のコードを追加します。

1-4-1. **[Tools] > [Language Templates]** を選択して、利用可能な **Language Templates** を開きます。

1-4-2. **[Verilog] > [Synthesis Constructs] > [Coding Examples] > [DSP] > [Squarer]** を展開します。

© Copyright 2026 Advanced Micro Devices, Inc.

1-4-3. プレビュー セクションから **squarer** 関数のコードをコピーし、**[Language Templates]** ダイアログ ボックスを閉じます。

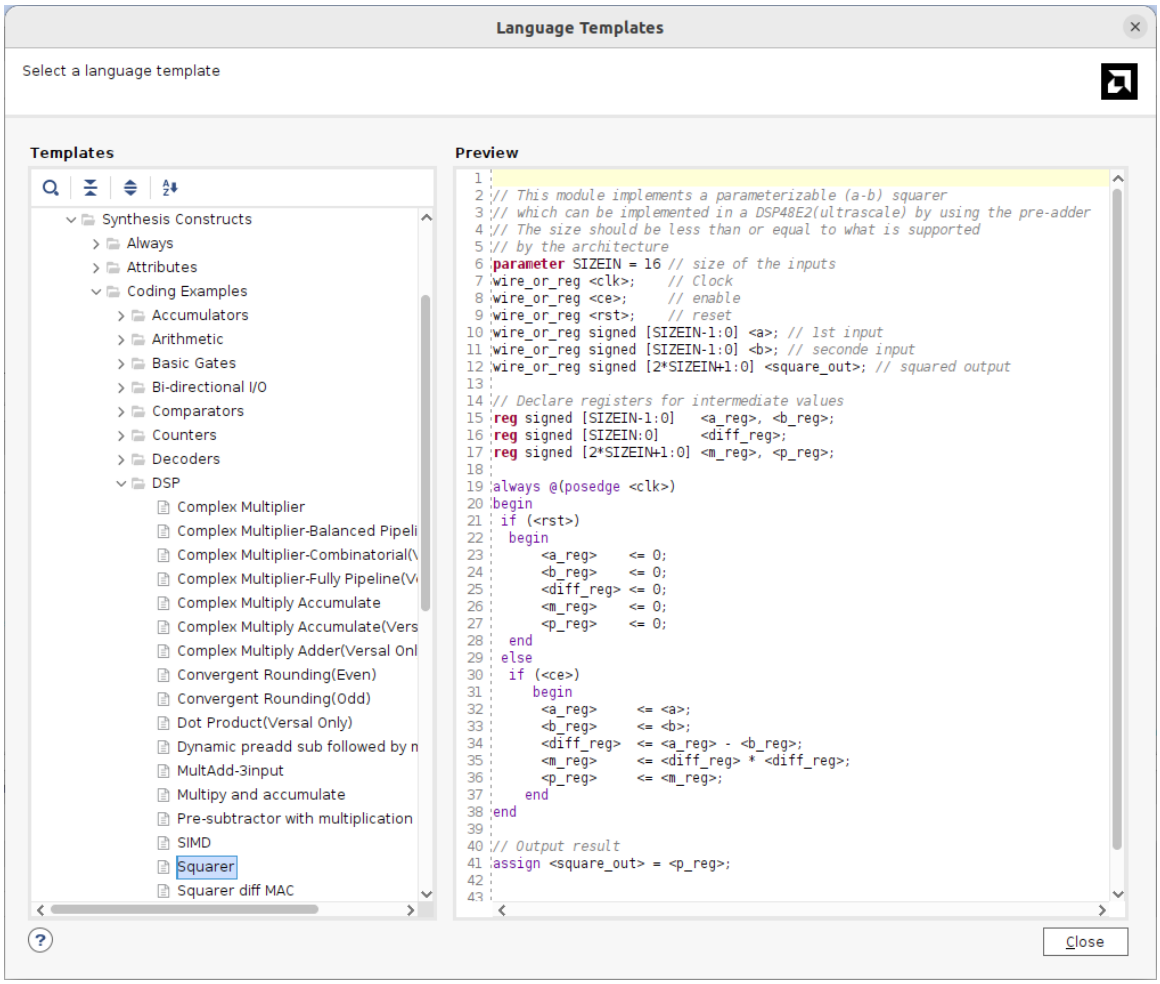


図 4-1: Language Templates - Squarer Function

1-4-4. **[Sources]** タブから **Squaring.v** ファイルを開きます。

1-4-5. **Squaring.v** ファイル内の「**paste the copied text from language template**」というコメントの下に、コピーしたコードを貼り付けます。

© Copyright 2026 Advanced Micro Devices, Inc.

1-4-6. コードを次のように変更します:

```
module Squaring #(parameter SIZEIN = 16)
(input clk, ce, rst,input [SIZEIN-1:0] a, b, output [2*SIZEIN+1:0] square_out  );
//Paste the copied text from Language Templates here
// Declare registers for intermediate values
reg signed [SIZEIN-1:0] a_reg, b_reg;
reg signed [SIZEIN:0] diff_reg;
reg signed [2*SIZEIN+1:0] m_reg, p_reg;

always @(posedge clk)
begin
 if (rst)
  begin
      a_reg    <= 0;
      b_reg    <= 0;
      diff_reg <= 0;
      m_reg    <= 0;
      p_reg    <= 0;
  end
 else
  if (ce)
     begin
      a_reg     <= a;
      b_reg     <= b;
      diff_reg  <= a_reg - b_reg;
      m_reg     <= diff_reg * diff_reg;
      p_reg     <= m_reg;
    end
end
// Output result
assign square_out = p_reg;
endmodule
```

図 4-2: DSP リソース用 2 乗関数コード

1-4-7. **[File] > [Text Editor] > [Save File]** を選択して、ファイルを保存します。

1-5. エラボレート済みデザインを開きます。

1-5-1. **[Flow Navigator]** の **[RTL ANALYSIS]** にある **[Open Elaborated Design]** をクリックして、デザインをエラボレートします。

エラボレート済み RTL デザインでは、[RTL Netlist]、[Schematic]、[Graphical Hierarchy] を含むさまざまな解析ビューを使用できます。これらのビューにはクロスセレクション機能があり、RTL のデバッグおよび最適化を行うことができます。

[Elaborate Design] ダイアログ ボックスが開きます。

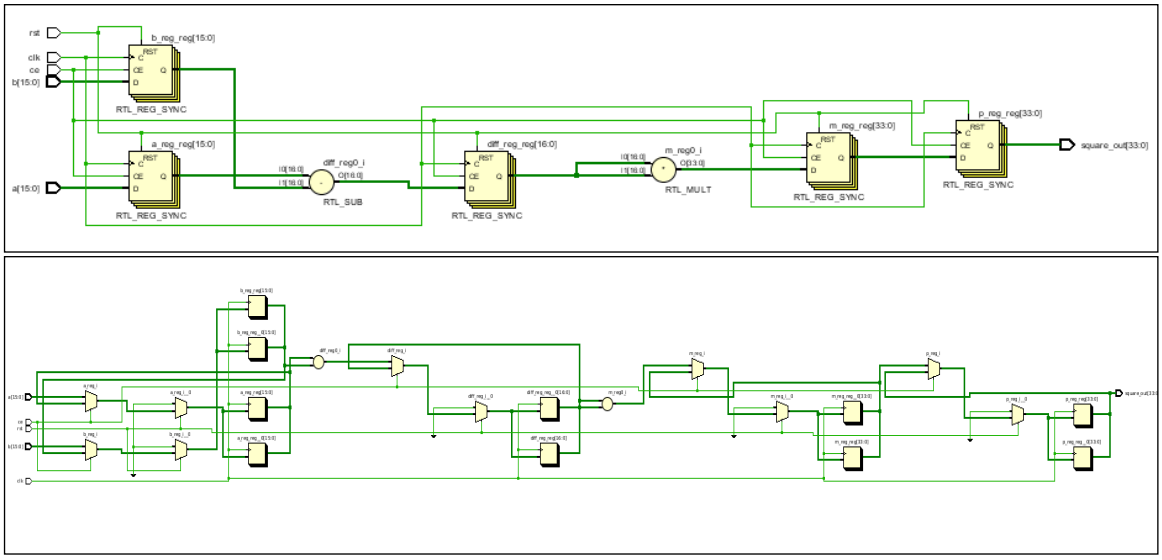
© Copyright 2026 Advanced Micro Devices, Inc.

1-5-2. ダイアログ ボックスで **[OK]** をクリックして、エラボレート デザインを開きます。

1-6. [RTL Schematic] を表示して、a_reg、b_reg、p_reg、および m_reg レジスタを確認します。

1-6-1. 前の手順で **[Schematic]** がデフォルトで開いていない場合は、**[RTL Analysis] > [Open Elaborated Design] > [Schematic]** を選択します。

これにより、トップレベル デザインの回路図が作成されます。



1-6-2. **[RTL schematic]** を閉じます。

1-7. エラボレート済みデザインを閉じます。

デザインの論理合成 手順 2

ここでは、デザインを論理合成し、DSP リソースを確認して、AREG、BREG、MREG、および PREG 属性の値を確認します。

2-1. デザインを論理合成します。

このタスクの実行方法を覚えていない場合は、Lab Reference Guide の Vivado Design Suite Operations にある「Synthesizing the Design」トピックを参照してください。

© Copyright 2026 Advanced Micro Devices, Inc.

2-2. 合成済みデザインを開きます。

このタスクの実行方法を覚えていない場合は、Lab Reference Guide の Vivado Design Suite Operations にある「Opening the Synthesized Design」トピックを参照してください。

2-3. AREG、BREG、MREG、PREG 属性の値を確認します。

2-3-1. **[Synthesis] > [Open Synthesized Design] > [Schematic]** を選択して、スケマティックを表示します。

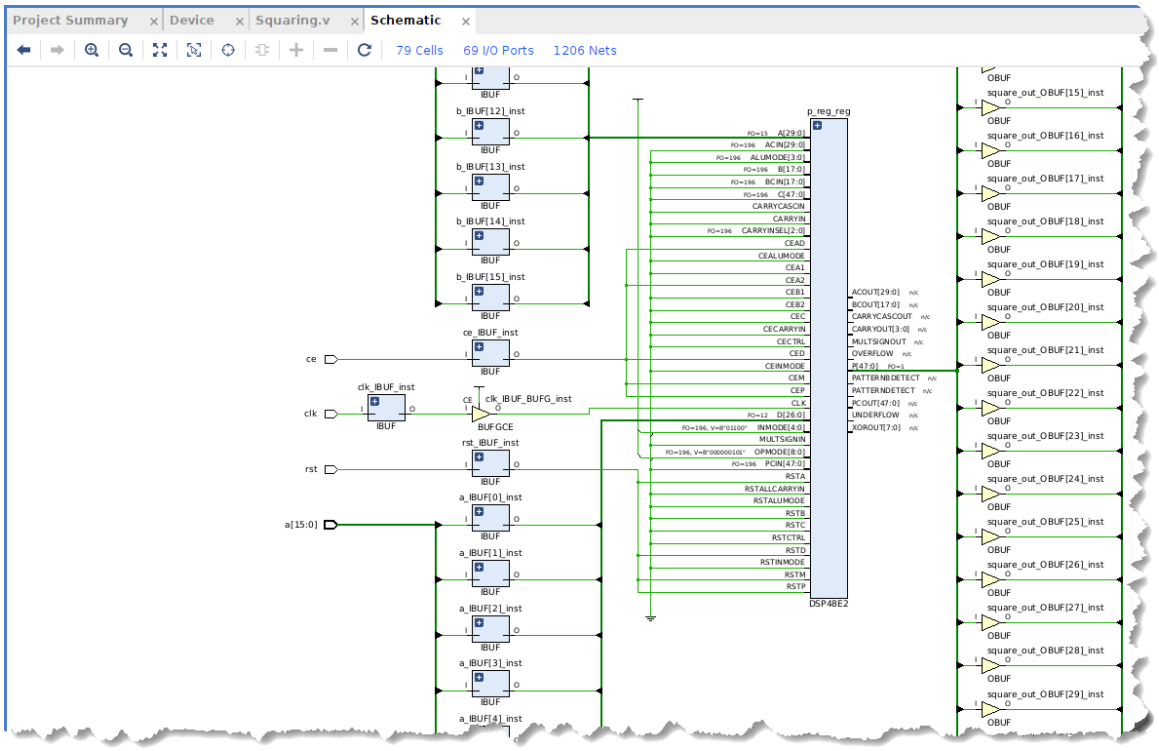


図 4-3: 合成済みデザインにおける DSP48E2

2-3-2. **p_reg_reg DSP** ブロックを検索して展開します。

2-3-3. **DSP_A_B_DATA**、**DSP_PREADD**、**DSP_PREADD_DATA**、**DSP_MULTIPLIER**、および **DSP_OUTPUT** ブロックを確認します。

2-3-4. ズームアウトして、**p_reg_reg** ブロック全体を選択します。

2-3-5. **p_reg_reg** ブロックを右クリックして **[Cell Properties]** を選択し、**[Cell Properties]** ペインを開きます。

© Copyright 2026 Advanced Micro Devices, Inc.
AMD together we advance_

2-3-6. **[Cell Properties]** ペインで **[Properties]** タブを選択します。

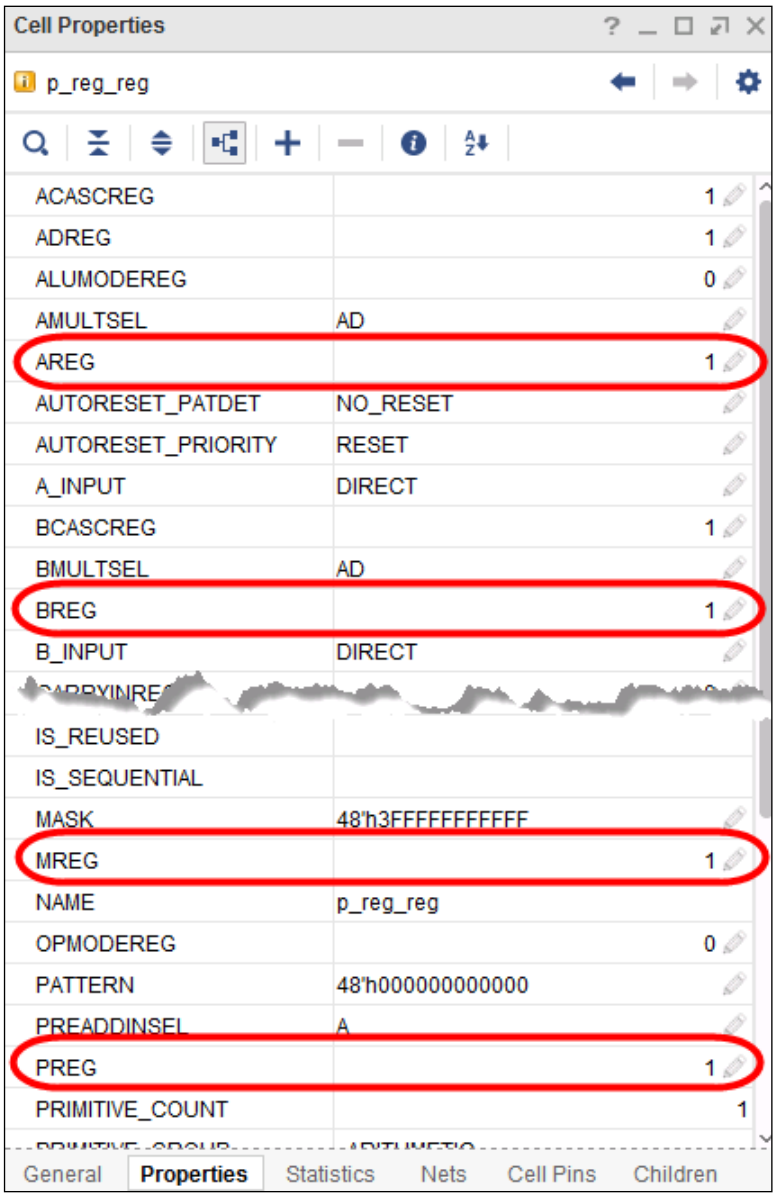


図 4-4: DSP48E2 Cell Properties

2-3-7. 各種属性の異なる値を確認します。

質問 1

AREG、BREG、MREG、および PREG 属性の値は何ですか。また、これらの値にはどのような意味がありますか。

© Copyright 2026 Advanced Micro Devices, Inc.

2-4. 合成済みデザインを閉じます。

2-5. Vivado Design Suite プロジェクトを閉じます。

このタスクの実行手順を覚えていない場合は、Lab Reference Guide の Vivado Design Suite Operations にある「Closing the Vivado Design Suite Project」トピックを参照してください。

一部のシステム (特に VM) では、メモリに制約がある場合があります。ワークスペースを削除するとディスク領域の一部が解放され、他の演習を実行できるようになります。

実行した演習を含むディレクトリは、グラフィカル インターフェイスまたはコマンドライン インターフェイスを使用して削除できます。どちらの方法を選択してもかまいません。どちらの処理でも、$TRAINING_PATH/dsp_resources ディレクトリ内のすべてのファイルが再帰的に削除されます。

2-6. [オプション] [ローカル VM のみ—CloudShare は対象外] ファイル システムをクリーンアップします。

GUI を使用する場合:

2-6-1. **$TRAINING_PATH/dsp_resources** に移動します。

2-6-2. **[dsp_resources]** を選択します。

2-6-3. **[Delete]** を押します。

-- または --

[Linux ユーザー]: コマンド ラインの使用:

2-6-4. **<Ctrl + Alt + T>** を押して **[Terminal]** ウィンドウを開きます。

2-6-5. 次のコマンドを入力して、ワークスペースの内容を削除します。

```
[host] $ rm -rf $TRAINING_PATH/dsp_resources
```


© Copyright 2026 Advanced Micro Devices, Inc.

まとめ

このラボでは、言語テンプレートを使用して二乗機能のデザインを作成しました。回路図を確認し、デザイン内で DSP リソースがどのように使用されているかを学習しました。また、DSP リソースのさまざまな属性についても確認しました。

解答

1. AREG、BREG、MREG、および PREG アトリビュートの値は何ですか。また、これらの値にはどのような意味がありますか。

   すべての AREG、BREG、MREG、および PREG 属性の値は 1 であり、この値は使用可能なパイプライン ステージ数を示します。

© Copyright 2026 Advanced Micro Devices, Inc.